HOSPITAL EMERGENCY QUEUE MANAGEMENT SYSTEM

Data Structures and Algorithms (DSA) Project Report

Team Members	Narendra Gangwar Samee Beg Tanishq
Programming Language	C
Core Data Structure	Priority-based queue implemented with an array
Institution	Invertis University

Prepared as an academic demonstration of data structures and queue operations for a hospital emergency waiting-list scenario.

1. Abstract

The Hospital Emergency Queue Management System is a menu-driven C program designed to organize a simulated hospital emergency waiting list. It registers patients, records basic details, assigns an urgency category, displays waiting patients, searches by patient ID, serves the next patient, and reports queue statistics. It demonstrates how a priority-based queue can order patients by urgency while preserving arrival order among patients with equal priority.

Important: This is an educational simulation only. In real healthcare, urgency must be assessed by qualified clinical staff using approved triage procedures. This program must not be used to make medical decisions.

2. Introduction

Emergency departments may receive patients with different levels of urgency. A simple first-come, first-served line may not represent these differences. A priority queue processes higher-priority items before lower-priority items. This project applies that concept to a simulated waiting list.

The program uses a C structure to store each patient's ID, name, age, condition, priority, and arrival order. An array stores waiting patients, while functions separate the main operations.

3. Problem Statement

To design and implement a menu-driven C program that maintains a waiting list, orders patients by an assigned urgency level, and provides registration, display, service, search, and statistics operations.

4. Objectives

• Register patients and assign each a unique ID.

• Store patient details in a structured record.

• Place higher-urgency patients before lower-urgency patients.

• Preserve arrival order when patients have equal priority.

• Display the waiting list and search by patient ID.

• Serve and remove the highest-priority patient.

• Display counts for Critical, Serious, and Normal categories.

• Demonstrate C structures, arrays, functions, loops, and searching.

5. Scope and Limitations

Scope

The program demonstrates core DSA concepts in a classroom setting. It supports up to 100 waiting patients during one run and uses a text-based menu.

Limitations

• Records are stored in memory and are lost when the program exits.

• There is no database, authentication, or graphical interface.

• Priority is entered by the operator; the program does not assess medical severity.

• The queue has a fixed capacity of 100 patients.

6. Technologies and Tools

Component	Purpose
C language	Program logic and menu-driven implementation
Structures	Store patient details as a single record

Component	Purpose
Array	Store waiting patients
Priority ordering	Arrange patients by urgency and arrival order
Functions	Separate registration, display, service, search, and statistics
GCC / compatible compiler	Compile and run the source file

7. Data Structure Used

Patient Structure

Each patient record stores:

• id: unique patient identifier generated by the program.

• name: patient's name.

• age: patient's age.

• condition: brief reason for the visit.

• priority: 1 = Critical, 2 = Serious, 3 = Normal.

• arrivalOrder: preserves arrival order for equal priorities.

Priority Queue Behaviour

A smaller numeric priority means greater urgency. A new patient is inserted before lower-urgency patients, but after patients with the same priority who arrived earlier. Serving a patient removes the first item from the ordered array.

8. Algorithm

1. Start and initialize the queue and ID counters.

2. Display the main menu.

3. Read the user's menu choice (1–6).

4. For choice 1, read patient details, assign priority and ID, then insert the patient in priority order.

5. For choice 2, display all waiting patients in queue order.

6. For choice 3, display and remove the first patient if the queue is not empty.

7. For choice 4, ask for a patient ID and search the waiting queue.

8. For choice 5, count and display patients in each priority category.

9. For choice 6, exit the program.

10. Repeat the menu until Exit is selected.

9. Flowchart

Main control flow: Start → Initialize variables and empty queue → Display menu → Read choice → Perform the selected operation → Display result → Check whether Exit was selected. If No, return to the menu; if Yes, Stop.

START
Initialize variables and empty queue
Display main menu and read choice
1 Register | 2 Display | 3 Serve | 4 Search | 5 Statistics | 6 Exit

Display operation result
Exit selected? — No: return to menu | Yes: STOP

10. Main Modules

Patient Registration

Reads name, age, and condition, asks for priority, generates a patient ID, and inserts the patient in priority order. Registration is refused if the queue is full.

Display Waiting Patients

Shows each waiting patient's ID, name, age, priority, and condition. An empty-queue message is shown when needed.

Serve Next Patient

Displays the first patient in the priority-ordered queue and removes that patient from the waiting list.

Search Patient by ID

Searches the current waiting list and displays details and queue position when the ID is found.

Queue Statistics

Counts and displays waiting patients in the Critical, Serious, and Normal categories, plus the total.

11. How to Compile and Run

Save the source code as **main.c** and ensure GCC or a compatible C compiler is installed.

Compile:

```
gcc main.c -o hospital_queue
```

Run on Windows:

```
hospital_queue.exe
```

Run on Linux:

```
./hospital_queue
```

12. Sample Test Plan

Test	Action	Expected result
1	Register a Critical patient	Patient receives an ID and is placed ahead of lower-priority patients.
2	Register equal-priority patients	Earlier arrival remains ahead of later arrival.
3	Display waiting patients	Patients appear in priority order.
4	Serve next patient	First patient is displayed and removed.
5	Search a waiting patient ID	Matching details are displayed.
6	Show statistics	Counts by priority and total are displayed.
7	Use an empty queue	Appropriate empty-queue message appears.

13. Advantages

• Simple menu-driven interface for beginners.

• Demonstrates practical use of structures, arrays, and functions.

• Places higher-urgency categories first.

• Maintains arrival order among equal-priority patients.

• Includes searching and queue statistics.

14. Future Enhancements

Future versions could store records permanently using file handling or a database, record registration timestamps, add user authentication and audit logs, provide a graphical interface, and improve input validation. Real clinical use would require professional oversight, privacy safeguards, thorough testing, and approved triage procedures.

15. Conclusion

The Hospital Emergency Queue Management System demonstrates how C programming and priority-based queue operations can organize a simulated waiting list. It includes registration, priority ordering, display, service, search, and statistics. The project reinforces fundamental DSA concepts and offers a base for further enhancement.

16. Team Members

• Narendra Gangwar

• Samee Beg

• Tanishq

Institution: Invertis University

Academic disclaimer: This report describes an educational prototype. It is not a medical device and must not be used to decide real patient priority.